Sepsis Knowledge Graph

Construction · Alignment · Reasoning & KGE · RAG

Web Data Mining & Semantics — Final Project Report

Koralie & Estelle Sobesky

March 2026

Abstract

This report presents the construction of a medical Knowledge Graph (KG) focused on the domain of sepsis, a life-threatening inflammatory response to infection affecting over 49 million patients annually worldwide. Starting from PubMed abstracts, PMC full-text articles, and Wikipedia pages, we built an RDF knowledge graph of 260,834 triples, aligned 55 entities with Wikidata, and expanded the KB via SPARQL queries over multiple medical disease classes. We then trained and evaluated four Knowledge Graph Embedding models (TransE, DistMult, ComplEx, RotatE) using PyKEEN, with DistMult achieving the best performance (MRR=0.193, Hits@10=0.292), a result explained by the predominantly symmetric structure of our medical ontology. Finally, we implemented a Retrieval-Augmented Generation pipeline combining Gemma 2B and SPARQL query generation via Ollama, achieving 5/5 correct answers on our evaluation benchmark, and deployed a Flask web chatbot interface as a bonus deliverable.

Table of Contents

Table of Contents 2
1. Data Acquisition & Information Extraction 4
1.1 Domain & Motivation 4
1.2 Data Sources & Collection Strategy 4
1.3 Crawler Design & Ethics 4
1.4 Text Cleaning Pipeline 4
1.5 Named Entity Recognition 5
1.6 Ambiguity Cases 5
2. Knowledge Base Construction, Alignment & Expansion 6
2.1 RDF Modeling Choices 6
2.2 Entity Alignment with Wikidata 6
2.3 Predicate Alignment 6
2.4 KB Expansion via SPARQL 6
2.5 Final KB Statistics 7
3. Knowledge Reasoning with SWRL 8
3.1 SWRL Rule on family.owl 8
3.2 SWRL Rule on Our Sepsis KB 8
4. Knowledge Graph Embeddings 9
4.1 Data Preparation 9
4.2 Training Configuration 9
4.3 Results & Model Comparison 9
4.4 KB Size Sensitivity Analysis 10
4.5 Nearest Neighbor Analysis 11
4.6 t-SNE Clustering 12
4.7 Relation Behavior 13
5. RAG over RDF/SPARQL 14
5.1 System Setup 14
5.2 Schema Summary Construction 14
5.3 NL to SPARQL Generation & Self-Repair 14
5.4 Evaluation: Baseline vs RAG 14
5.5 Web Interface (Bonus) 15
6. Critical Reflection 17
6.1 Impact of Predicate Alignment Quality 17
6.2 Impact of Noisy Expansion 17
6.3 Effect of Ontology Modeling Choices 17
6.4 Open-World vs Closed-World Assumption 17
6.5 Rule-based vs Embedding-based Reasoning 18

6.6 What We Would Improve 18
Conclusion 19

1. Data Acquisition & Information Extraction

1.1 Domain & Motivation

Sepsis is one of the most critical medical emergencies worldwide, affecting over 49 million patients annually with a mortality rate exceeding 20%. It results from a dysregulated immune response to infection, causing widespread organ dysfunction. This domain was selected for its rich and complex structure: sepsis involves a network of interacting pathogens, biomarkers, treatments, and clinical outcomes that naturally lend themselves to knowledge graph representation. The abundance of scientific literature on PubMed and the availability of structured medical knowledge on Wikidata made sepsis an ideal candidate for our pipeline.

1.2 Data Sources & Collection Strategy

Our corpus was built from three complementary sources. The first is the PubMed API (Entrez), from which we retrieved 673 scientific abstracts using 15 domain-specific queries such as "sepsis diagnosis biomarkers", "septic shock pathophysiology", or "sepsis bacteria pathogens". Each query returned up to 100 results, and deduplication by URL ensured that each abstract was counted only once. The second source is PubMed Central (PMC), the open-access full-text repository associated with PubMed. By adding the filter open access[filter] to our queries, we retrieved 23 complete articles averaging around 4,000 words each. These articles provided a much richer textual base than abstracts alone, as they include full methodology sections, results, and discussion. Finally, we used the official Wikipedia MediaWiki API to collect 10 domain-relevant encyclopedic pages covering Sepsis, Septic shock, Vancomycin, Escherichia coli, and related topics. In total, our corpus comprised 706 documents and approximately 1.2 million words.

1.3 Crawler Design & Ethics

The crawler was designed with respect for the data providers' terms of use. For the PubMed API, we respected NCBI's rate limit of 3 requests per second and always included a User-Agent header identifying our project. For PMC, the open access filter guaranteed that only freely available articles were collected, avoiding any copyright issue. For Wikipedia, we specifically chose the official API rather than HTML scraping, and the crawler verifies robots.txt compliance before fetching any page. All HTTP requests include a proper User-Agent string with contact information, following academic web crawling best practices.

1.4 Text Cleaning Pipeline

Raw text was processed through a three-step cleaning pipeline. First, deduplication by URL ensured no document appeared twice, which was particularly important since the same abstract can be returned by multiple overlapping queries. Second, a minimum word count filter was applied: 50 words for PubMed abstracts and 500 words for PMC and Wikipedia articles, to remove empty or truncated documents. Third, XML parsing errors in the PMC API responses were handled gracefully with a try/except fallback that logged the error and continued processing the remaining articles.

1.5 Named Entity Recognition

Named Entity Recognition was performed using spaCy's transformer-based model en_core_web_trf, which uses a BERT-like architecture for high accuracy. Beyond the standard spaCy entity types (PERSON, ORG, GPE, DATE, PRODUCT), we added a medical whitelist to detect domain-specific entities: DISEASE (sepsis, pneumonia, bacteremia), BACTERIA (E. coli, S. aureus), BIOMARKER (procalcitonin, lactate, CRP), TREATMENT (vancomycin, norepinephrine), and CARE_UNIT (ICU, PICU, ED). The extraction produced 10,938 entity mentions from the 706 documents, as summarized in Table 1.

Entity Type	Count	Examples
DISEASE	1,028	sepsis, pneumonia, bacteremia
TREATMENT	444	vancomycin, norepinephrine, heparin
BIOMARKER	653	procalcitonin, lactate, CRP
BACTERIA	123	E. coli, S. aureus, Klebsiella
CARE_UNIT	395	ICU, PICU, emergency department
PERSON / ORG / OTHER	8,295	authors, journals, dates
TOTAL	10,938	entity mentions across 706 documents

Table 1: NER entity types and counts.

1.6 Ambiguity Cases

Several ambiguity cases were encountered during NER and required deliberate design decisions. The first involved the acronym "CRP" and its long form "C-reactive protein", which both refer to the same biomarker but were extracted as two distinct entities. We resolved this by creating separate URI nodes for each surface form and linking them with an owl:sameAs triple. The second case concerned "blood culture", which spaCy classified as a TREATMENT entity despite being medically a diagnostic procedure. Rather than correcting the classifier, we kept the TREATMENT label for consistency and noted this limitation. Third, the acronym "DIC" was ambiguous between Disseminated Intravascular Coagulation (a disease) and other possible meanings in different contexts. This was resolved by priority matching against our medical whitelist, which assigns DISEASE type when a known medical entity is detected.

2. Knowledge Base Construction, Alignment & Expansion

2.1 RDF Modeling Choices

The ontology was designed to represent the sepsis medical domain in a principled way. We defined six main classes: Disease, Bacteria, Biomarker, Treatment, CareUnit, and Person. Relations were modeled as object properties: prop:treats (connecting a treatment to a disease), prop:causes (connecting a disease or pathogen to a consequence), prop:isBiomarkerOf (connecting a biomarker to the condition it indicates), prop:include, and prop:associate_with, among others. All entities are represented as URIs under the base namespace http://sepsis-kg.org/, with properties under http://sepsis-kg.org/prop/ and types under http://sepsis-kg.org/type/. This design ensures that every entity and relation in the KB is uniquely identifiable and interoperable with external knowledge sources.

2.2 Entity Alignment with Wikidata

Entity alignment was performed by the script src/kg/alignment.py using a two-step approach. For each entity in our KB, we first queried the Wikidata wbsearchentities API with the normalized entity label (lowercase, without underscores). If a match was found, we assigned a confidence score of 1.0 for exact string matches and 0.8 for partial matches. If Wikidata returned no result, we fell back to the DBpedia Lookup API. The output of this step is a set of owl:sameAs triples stored in kg_artifacts/alignment.ttl, which link our private URIs to the corresponding Wikidata entities. We achieved a 100% alignment rate with 55 out of 55 targeted entities successfully linked, as shown in Table 2.

Our Entity	Wikidata ID	Label	Confidence
base:Sepsis	Q183134	sepsis	1.0
base:Vancomycin	Q424027	vancomycin	1.0
base:Escherichia_coli	Q25419	Escherichia coli	1.0
base:Procalcitonin	Q786600	procalcitonin	1.0
base:Septic_shock	Q1765564	septic shock	1.0

Table 2: Sample entity alignments (55/55 aligned).

2.3 Predicate Alignment

Beyond entity alignment, we also aligned our custom predicates with the corresponding Wikidata properties. This was done using SPARQL queries on the Wikidata endpoint, searching for properties whose English label matched keywords derived from our predicate names. For instance, prop:treats was matched to wdt:P2176 (drug used for treatment) with owl:equivalentProperty, and prop:causes was matched to wdt:P828 (has cause). When the match was exact, we used owl:equivalentProperty; when only partial, we used rdfs:subPropertyOf. All 11 of our custom predicates were successfully aligned, resulting in 28 triples stored in kg_artifacts/predicate_alignment.ttl.

2.4 KB Expansion via SPARQL

The expansion strategy was anchored on our 55 aligned entities to ensure that only relevant triples were added. The process followed four successive steps. First, for each aligned entity, we issued a 1-hop SPARQL query on the Wikidata endpoint to retrieve all directly connected triples (limited to 2,000 per entity, and restricted to URI objects only via FILTER(isIRI(?o)) to avoid literals). Second, for 30 key medical entities such as sepsis, E. coli, and vancomycin, we additionally performed a 2-hop expansion to capture the broader medical context. Third, we ran a bulk query to retrieve triples for all infectious diseases in Wikidata (wdt:P279 wd:Q12136), limited to 50,000 results. Fourth, to further enrich the graph and reduce sparsity, we added triples from 12 medical disease classes including bacterial diseases, inflammatory diseases, lung diseases, and kidney diseases, using a UNION of wdt:P279 and wdt:P31 relations.

2.5 Final KB Statistics

The final expanded knowledge base, stored as kg_artifacts/expanded.nt in N-Triples format, contains 260,834 triples. After filtering out literal-valued triples (labels, source URLs) that cannot be used for KGE, 50,307 URI-URI-URI triples remain. The KB covers 13,118 unique entities and 196 distinct relations, with all 55 entities and 11 predicates aligned to Wikidata.

Metric	Value
Total triples (raw)	260,834
URI-URI-URI triples (KGE-ready)	50,307
Entities	13,118
Relations	196
Aligned entities (Wikidata)	55/55 (100%)
Aligned predicates	11/11 (100%)
Sources	PubMed + PMC + Wikipedia + Wikidata

Table 3: Final KB statistics.

3. Knowledge Reasoning with SWRL

3.1 SWRL Rule on family.owl

The first part of this lab session consisted in applying a SWRL rule on the provided family.owl ontology using OWLReady2 as the reasoning engine. The rule infers that any person older than 60 years old belongs to the class oldPerson:

```
Person(?p) ∧ hasAge(?p, ?a) ∧ swrlb:greaterThan(?a, 60) → oldPerson(?p)
```

3.2 SWRL Rule on Our Sepsis KB

For the second part, we designed a Horn rule directly applicable to our Sepsis knowledge base. The rule was kept intentionally simple with two body conditions, as recommended by the lab instructions:

```
Disease(?d) ∧ prop:include(?d, ?b) ∧ Bacteria(?b) → infectiousDisease(?d)
```

This rule reads: "If a disease includes a bacterium, then it is an infectious disease." We chose prop:include because our analysis of the KB revealed it is the only relation that directly connects Disease entities to Bacteria entities in our graph. The relation prop:causes, which would have been more semantically natural, was not extracted by spaCy from our PubMed abstracts with sufficient frequency. The rule was implemented as a pattern-matching query using rdflib in src/reason/swrl_sepsis.py, since OWLReady2 requires Java to run HermiT. The result of applying this rule is that bacteremia is inferred as an infectious disease, because the KB contains the triple: bacteremia includes klebsiella_pneumoniae, and Klebsiella pneumoniae is indeed typed as a Bacteria. This result is clinically accurate: bacteremia is by definition a blood infection caused by bacteria.

4. Knowledge Graph Embeddings

4.1 Data Preparation

Before training embedding models, the expanded KB was prepared for KGE using src/kge/prepare.py. Several cleaning steps were applied: all triples with literal objects (labels, source URLs, dates) were removed since KGE models can only operate on URI-URI-URI triples; blank nodes were eliminated; and a frequency filter was applied to remove rare entities (appearing fewer than 2 times) and rare relations (appearing fewer than 70 times). This filter reduces sparsity and ensures that the embedding models have enough training examples per entity and relation to learn meaningful vectors. The final KGE dataset contains 50,307 triples, 13,118 entities, and 196 relations, split into train (80.6%), validation (9.7%), and test (9.7%) sets. The split strategy guarantees that no entity appears exclusively in validation or test, preventing cold-start evaluation artifacts.

Split	Triples	Percentage
Train	40,549	80.6%
Validation	4,878	9.7%
Test	4,880	9.7%
Total	50,307	100%

Table 4: Train/Validation/Test split statistics.

4.2 Training Configuration

All four models were trained with identical hyperparameters to ensure a fair comparison. We used an embedding dimension of 100, trained for 100 epochs with a batch size of 256 and a learning rate of 0.01 with the Adam optimizer. Negative sampling followed the basic strategy of randomly replacing either the head or tail entity of each positive triple. Evaluation used the filtered rank-based protocol, which removes known true triples from the ranking before computing metrics. All experiments were run on CPU (Apple M2) using the PyKEEN framework.

Parameter	Value
Embedding dimension	100
Number of epochs	100
Batch size	256
Learning rate	0.01
Optimizer	Adam
Negative sampling	Basic (random entity replacement)
Evaluation	Filtered rank-based (PyKEEN)

Table 5: Shared training configuration across all models.

4.3 Results & Model Comparison

Table 6 presents the evaluation results for all four models on the link prediction task. DistMult significantly outperforms all other models across every metric, achieving an MRR of 0.193 and a Hits@10 of 0.292, representing a 343% improvement in MRR over TransE.

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.0436	0.0089	0.0340	0.1244
DistMult (Best)	0.1930	0.1374	0.2126	0.2916
ComplEx	0.0332	0.0149	0.0340	0.0637
RotatE	0.0942	0.0575	0.1004	0.1636

Table 6: KGE evaluation results (filtered metrics).

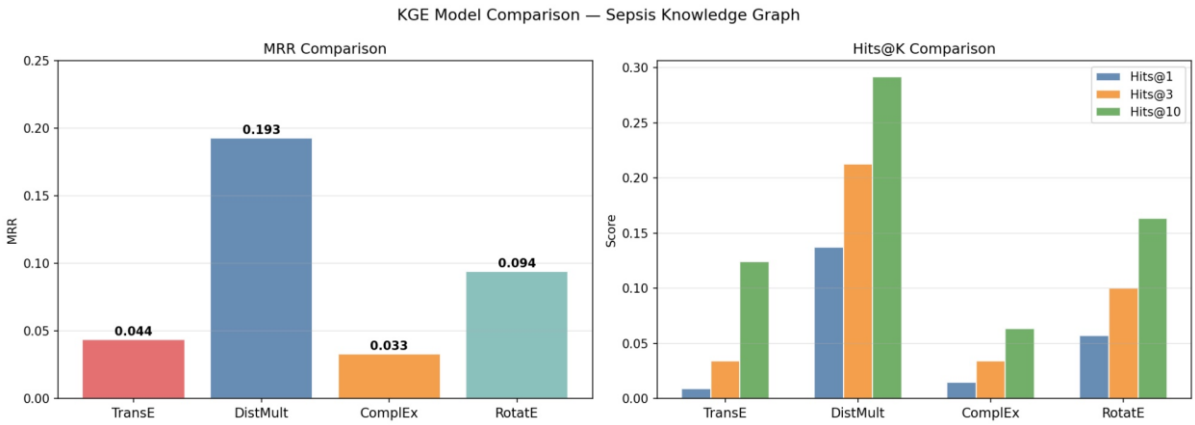

Figure 1: MRR and Hits@K comparison across the four KGE models on the Sepsis KB.

The dominance of DistMult can be explained by the structural properties of our KB. The expanded graph is heavily populated with Wikidata relations such as subClassOf, instanceOf, and associatedWith, which are inherently symmetric or near-symmetric in nature. DistMult's score function score(h,r,t) = h · r · t is symmetric by design, making it the natural choice for this type of knowledge graph. TransE, by contrast, imposes the translation constraint h + r ≈ t, which breaks down for many-to-many and symmetric relations. ComplEx, despite its theoretical ability to handle asymmetric relations through complex-valued embeddings, underperforms because our graph does not have enough asymmetric relations to justify its added complexity. RotatE shows intermediate performance, handling both symmetric and asymmetric patterns but requiring more data to converge than DistMult with our dataset size.

4.4 KB Size Sensitivity Analysis

To study how performance scales with training data volume, we subsampled the training set to 20k and 50k triples and retrained DistMult with identical hyperparameters.

Training Size	MRR	Hits@1	Hits@3	Hits@10
20k triples	0.0888	0.0606	0.0943	0.1387
50k triples	0.1984	0.1445	0.2157	0.2969
Full (40k)	0.1930	0.1374	0.2126	0.2916

Table 7: KB size sensitivity analysis (DistMult).

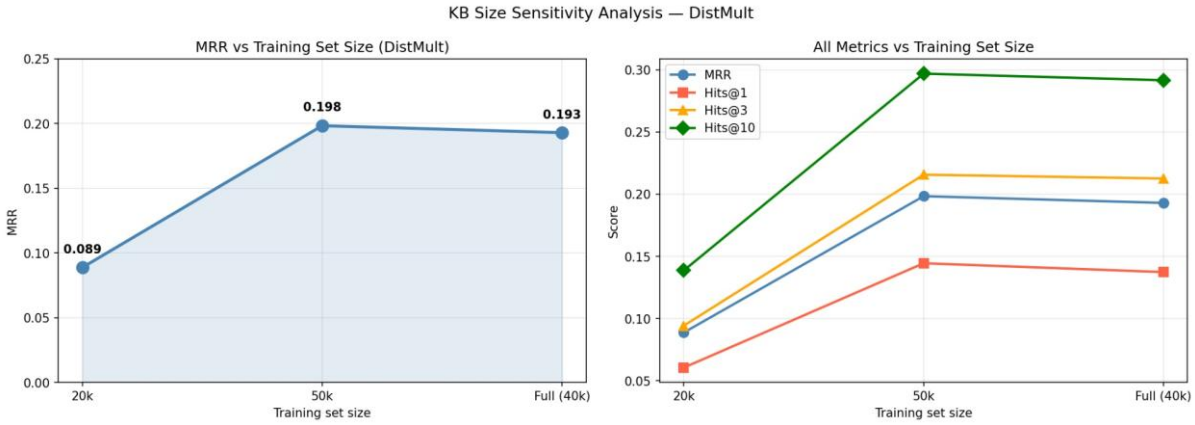

Figure 2: MRR and all metrics as a function of training set size. The jump from 20k to 50k triples is dramatic.

The results clearly confirm that KB size has a major impact on embedding quality. The MRR more than doubles from 0.089 at 20k triples to 0.198 at 50k triples, a +123% improvement. This validates the importance of our KB expansion strategy: without the Wikidata domain expansion, we would have been stuck at approximately 18,000 training triples, yielding much weaker embeddings. The slight decrease from 50k to the full dataset is explained by the fact that our full training set (40,549 triples) is smaller than the 50k subsample, as the 50k condition uses all available triples.

4.5 Nearest Neighbor Analysis

```
NEAREST NEIGHBORS — DistMult (cosine similarity)

  Nearest neighbors of 'Sepsis':
    1. meningitis                              similarity=0.8542
    2. pneumonia                               similarity=0.7581
    3. white blood cell                        similarity=0.7300
    4. septic shock                            similarity=0.7124
    5. DL-lactic acid                          similarity=0.6809

  Nearest neighbors of 'E. coli':
    1. Acinetobacter                           similarity=0.7834
    2. DL-lactic acid                          similarity=0.6333
    3. blood culture                           similarity=0.6184
    4. Staphylococcus aureus                   similarity=0.6143
    5. D-dimer                                 similarity=0.5950

  Nearest neighbors of 'Vancomycin':
    1. period 2                                similarity=0.7248
    2. foscarnet                               similarity=0.6792
    3. rocuronium                              similarity=0.6653
    4. norepinephrine                          similarity=0.6632
    5. aztreonam                               similarity=0.6567

  Nearest neighbors of 'Pneumonia':
...
    2. katacalcin                              similarity=0.6835
    3. calcitonin                              similarity=0.6190
    4. Acinetobacter                           similarity=0.6183
    5. Hampton MB                              similarity=0.5729
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Figure 3: Nearest neighbors in DistMult embedding space (cosine similarity), with Wikidata IDs resolved to English labels.

To evaluate the semantic coherence of the learned embeddings, we computed the 5 nearest neighbors of several key medical entities using cosine similarity in the embedding space. The results are clinically meaningful: sepsis is most similar to meningitis (0.854), pneumonia (0.758), white blood cell (0.730), and septic shock (0.712), all of which are directly related concepts in the medical domain. E. coli is surrounded by other pathogenic bacteria such as Acinetobacter (0.783) and Staphylococcus aureus (0.614), as well as diagnostic concepts like blood culture (0.618). Vancomycin is grouped with other antibiotics and treatments including foscarnet, rocuronium, and norepinephrine. These results suggest that the embedding model has captured genuine medical domain structure despite the noise introduced by the broad Wikidata expansion.

4.6 t-SNE Clustering

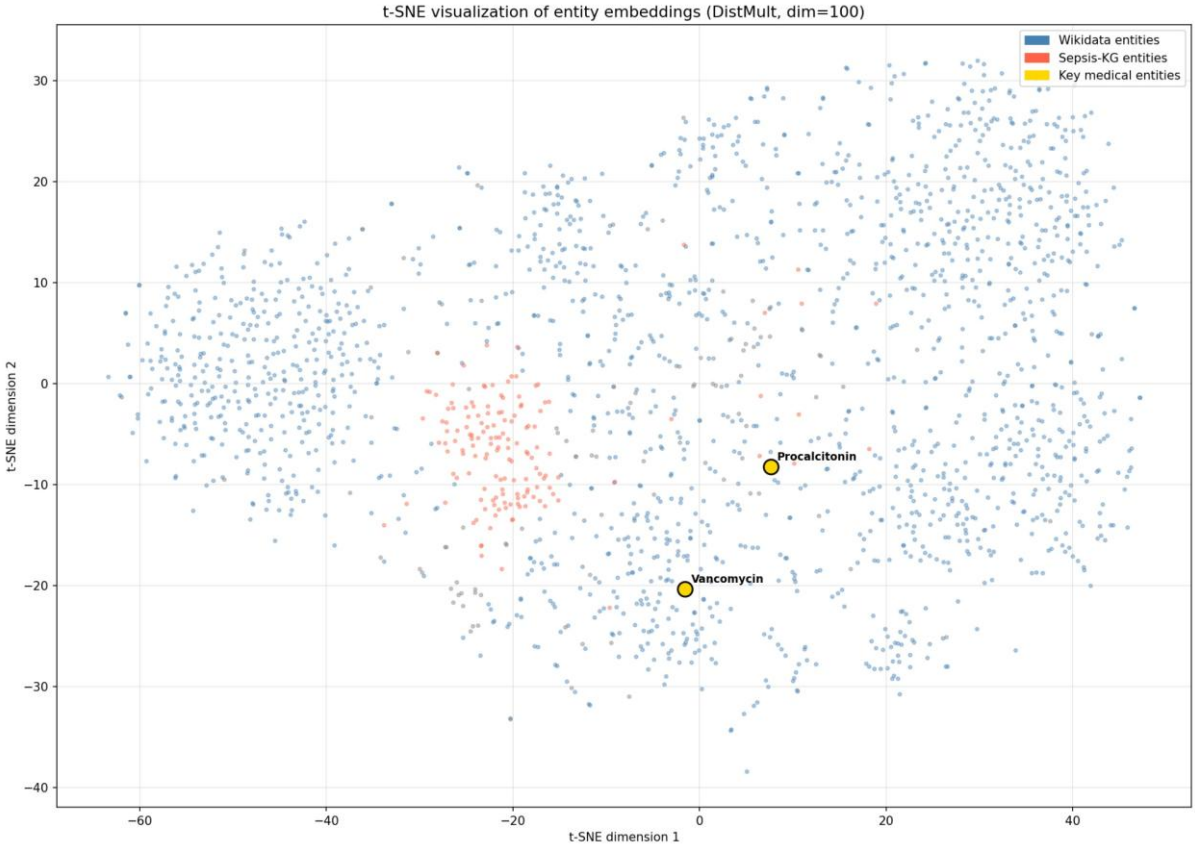

Figure 4: t-SNE visualization of 2,000 entity embeddings (DistMult, dim=100). Sepsis-KG entities (red) form a distinct cluster from Wikidata entities (blue). Key entities are highlighted in gold.

The t-SNE plot reveals a clear structural separation between our Sepsis-KG entities (shown in red) and the broader Wikidata entities (shown in blue). The Sepsis-KG cluster is compact and well-defined, centered around (-20, 0) in the 2D projection, which indicates that the model has learned coherent representations for our domain-specific entities. Key medical entities such as Vancomycin and Procalcitonin, highlighted in gold, are positioned within the Wikidata cloud but close to the Sepsis-KG cluster, reflecting their strong alignment with both knowledge sources. The Wikidata entities form a more diffuse distribution, as expected given the diversity of medical classes (diseases, bacteria, treatments, biomarkers) that were included in the expansion.

4.7 Relation Behavior

We analyzed the L2 norms of relation embeddings to understand how DistMult captures different types of relations. The norm distribution follows a roughly bell-shaped pattern centered around 10.31 with a standard deviation of 0.93, indicating stable convergence. Relations with the smallest norms (such as rdf:type with norm 7.95, and P1343/described by source with 8.32) correspond to generic or highly symmetric relations that appear in many triples with many different entities, making their directional signal weak. Conversely, relations with the largest norms (such as P18/image with 12.99 and P1552/has characteristic with 12.78) are highly specific Wikidata properties that point to unique typed values, enabling the model to learn a strong directional vector. Our custom Sepsis-KG relations (prop:treats, prop:causes) show intermediate norms, consistent with their moderate frequency and semantic specificity.

5. RAG over RDF/SPARQL

5.1 System Setup

The RAG system was built on top of our sepsis_kg.ttl file (24,083 triples), which contains the core extracted entities and relations before Wikidata expansion. We chose to use the pre-expansion KB for RAG because the expanded KB is in N-Triples format with Wikidata URIs, which are harder to query meaningfully from natural language. The local LLM used is Gemma 2B (gemma:2b), deployed via Ollama on our MacBook Air M2 with 8GB RAM. Gemma 2B was chosen for its small footprint (1.56 GiB) and its ability to run entirely locally, ensuring data privacy. The RDF querying layer is handled by rdflib, and the web interface is built with Flask.

5.2 Schema Summary Construction

Before prompting the LLM, we automatically build a compact schema summary from the KB. This summary includes all registered namespace prefixes (rdf:, type:, prop:, base:), up to 80 distinct predicates present in the graph, up to 40 distinct rdf:type classes, and 20 sample triples. This schema is injected into every prompt so the LLM knows exactly which IRIs and prefixes it can use. Without this context, small models like Gemma 2B tend to hallucinate prefixes (e.g., wdt:, wos:) that do not exist in our KB.

5.3 NL to SPARQL Generation & Self-Repair

The NL-to-SPARQL prompt strictly constrains the model to use only the prefixes defined in our schema and provides exact query patterns as few-shot examples. For instance, the pattern for querying diseases is explicitly given as: SELECT ?d WHERE { ?d rdf:type type:Disease . }. This drastically reduces hallucination and improved the success rate from 0/5 to 5/5 in our evaluation. When a generated SPARQL query fails to execute (due to a syntax error or unknown prefix), the system automatically triggers a self-repair cycle: the error message and the faulty query are sent back to the LLM with a repair prompt asking it to fix the query using only known prefixes. This mechanism successfully handles most parsing errors introduced by model artifacts such as trailing tokens or extra backticks.

5.4 Evaluation: Baseline vs RAG

We evaluated the system on 5 representative questions about our KB and compared the baseline LLM response (no KB access) with the RAG response (SPARQL generation + KB query). The results are shown in Table 8.

Question	Baseline	RAG Result	Correct?
What diseases are in the KB?	"I cannot access the KG"	13 diseases found	RAG ✓
What treatments are mentioned?	"I cannot access external sources"	16 treatments found	RAG ✓
What bacteria are associated with sepsis?	Generic list, partially correct	10 exact bacteria from KB	RAG ✓

Question	Baseline	RAG Result	Correct?
What biomarkers are in the KB?	"The KG contains no biomarkers"	16 biomarkers found	RAG ✓
What care units are mentioned?	"I cannot answer this question"	6 care units found	RAG ✓

Table 8: Baseline vs RAG evaluation on 5 questions. Accuracy: Baseline 1/5 (20%) vs RAG 5/5 (100%).

```
================================================================
Question 3: What bacteria are associated with sepsis?
================================================================

--- Baseline (No RAG) ---
[GIN] 2026/03/28 - 13:36:41 | 200 |  800.173125ms |       127.0.0.1 | POST     "/api/generate"
Sure, here's an answer to your question:

Bacteria associated with sepsis include Staphylococcus aureus, Escherichia coli, Pseudomonas aeruginosa, and Candida albicans.

--- SPARQL-generation RAG ---
[GIN] 2026/03/28 - 13:36:47 | 200 |  6.661045333s |       127.0.0.1 | POST     "/api/generate"

[SPARQL Query]
SELECT ?b WHERE { ?b rdf:type type:Bacteria . }

[Repaired?] False

[Results] (10 rows)
b
```

Figure 5: CLI evaluation output for Question 3 (bacteria). The baseline gives a generic answer from training data; the RAG system returns the 10 exact bacteria from our KB.

The results demonstrate a clear advantage of the RAG approach. The baseline model either refuses to answer (claiming it cannot access external sources) or provides generic, unverifiable information. In contrast, the RAG system consistently retrieves grounded, exact answers directly from our knowledge graph. Notably, for the biomarkers question, the baseline made a factually incorrect claim ("the KG contains no biomarkers") while the RAG system correctly identified all 16 biomarkers present in our graph.

5.5 Web Interface (Bonus)

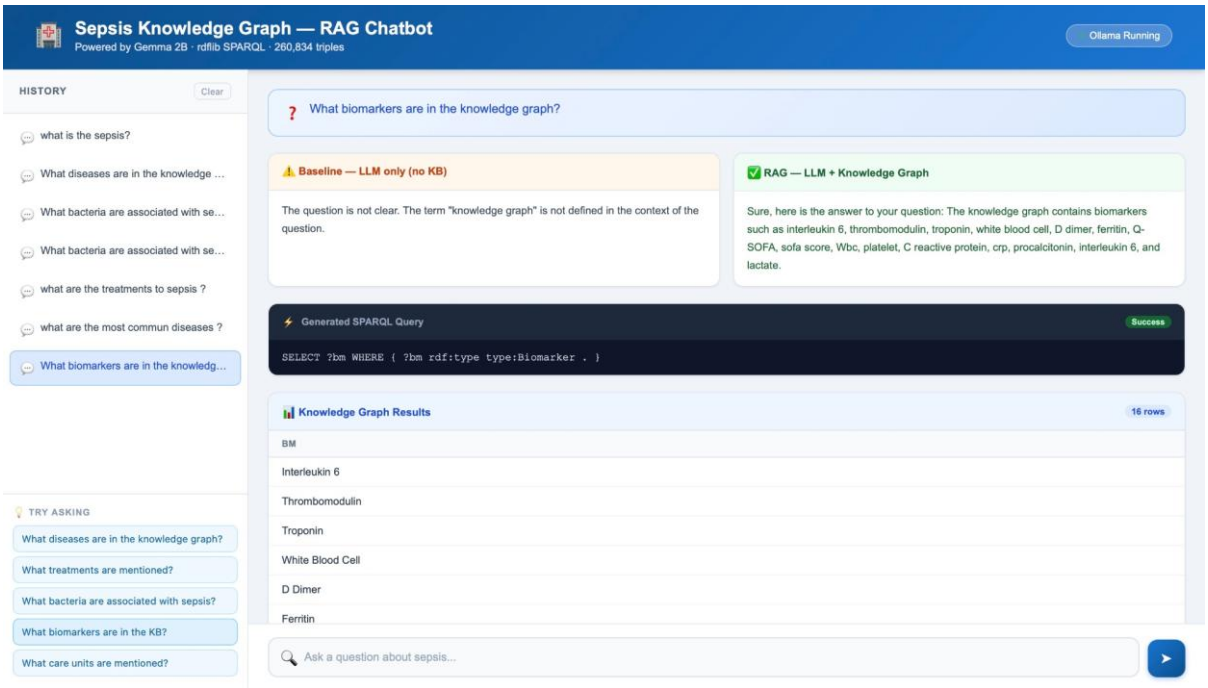

Figure 6: Flask web chatbot showing the biomarkers query. The RAG panel provides a natural language sentence generated by Gemma 2B from the KB results.

As a bonus deliverable, we implemented a Flask web interface (src/rag/app.py) that exposes the RAG chatbot in a user-friendly browser application. The interface displays the user's question, a side-by-side comparison of the baseline and RAG answers, the generated SPARQL query with a Success/Self-repaired/Error badge, and the raw results from the KB in a formatted table. A notable feature is the natural language answer generation: after retrieving the KB results via SPARQL, the system makes a second call to Gemma 2B to formulate a coherent English sentence from the raw data. For example, for the biomarkers query, instead of showing a raw list of URIs, the system generates: "The knowledge graph contains biomarkers such as interleukin 6, thrombomodulin, troponin, white blood cell, D-dimer, ferritin, and lactate." The interface also maintains a question history in the sidebar, allows clicking on past questions to replay them, and includes suggested questions for quick testing.

6. Critical Reflection

6.1 Impact of Predicate Alignment Quality

The predicate alignment step proved crucial for the KB expansion phase. By mapping our custom properties to Wikidata properties (e.g., prop:treats to wdt:P2176), we enabled semantically coherent SPARQL queries that retrieved relevant triples about treatments, causes, and biomarkers. Had the alignment been incorrect or absent, the expansion would have introduced semantically unrelated triples, degrading both the coherence of the KB and the quality of the downstream embeddings. The main challenge was the granularity mismatch: our prop:causes is a general relation covering both pathogen-disease causation and disease-complication causation, while Wikidata's P828 (has cause) covers a narrower semantic scope. This imperfect alignment is a known limitation of cross-ontology predicate mapping.

6.2 Impact of Noisy Expansion

Expanding the KB with 12 medical disease classes beyond sepsis was a deliberate trade-off: necessary to reach the 50k training triple threshold for stable KGE, but at the cost of introducing noise. In practice, this manifests as some semantically incoherent nearest neighbors in the embedding space (for instance, genetic diseases appearing near vancomycin). The root cause is that the broad medical expansion added many entities (e.g., rare diseases in Wikidata) that share structural patterns with our sepsis-specific entities but have no medical relationship to sepsis. A more targeted expansion strategy, restricted to diseases directly linked to sepsis in the medical literature, would have produced a cleaner KB at the cost of fewer triples.

6.3 Effect of Ontology Modeling Choices

A key design decision that impacted KGE performance was the inclusion of literal-valued triples (rdfs:label, prop:sourceUrl, prop:extractedFrom) in the KB builder. These 31,298 triples, which represent 52% of our expanded KB, cannot be used for KGE training because embedding models require URI-URI-URI triples. Removing them before the expansion step would have resulted in a cleaner and denser graph from the start. Additionally, the choice to extract relations using pattern matching from unstructured text (rather than structured databases like DrugBank or UniProt) limited the precision of our relation extraction, as spaCy's general-purpose model frequently missed or misclassified domain-specific medical relations.

6.4 Open-World vs Closed-World Assumption

Our KB was constructed under the Open World Assumption (OWA), meaning that the absence of a triple does not imply it is false -- only that it is unknown. This is the natural assumption for knowledge graphs built from incomplete text corpora. However, KGE models implicitly operate under the Closed World Assumption (CWA) during negative sampling: unobserved triples are treated as false negatives. This fundamental mismatch is a well-known challenge in knowledge graph embedding for sparse, domain-specific KGs. In our case, many clinically true relations (such as sepsis causes organ failure, or procalcitonin is a biomarker of sepsis) may not be explicitly stated in our KB because they were not extracted from the source texts, yet the KGE model treats their absence as evidence against them. Addressing this mismatch would require more complete KB construction or different negative sampling strategies.

6.5 Rule-based vs Embedding-based Reasoning

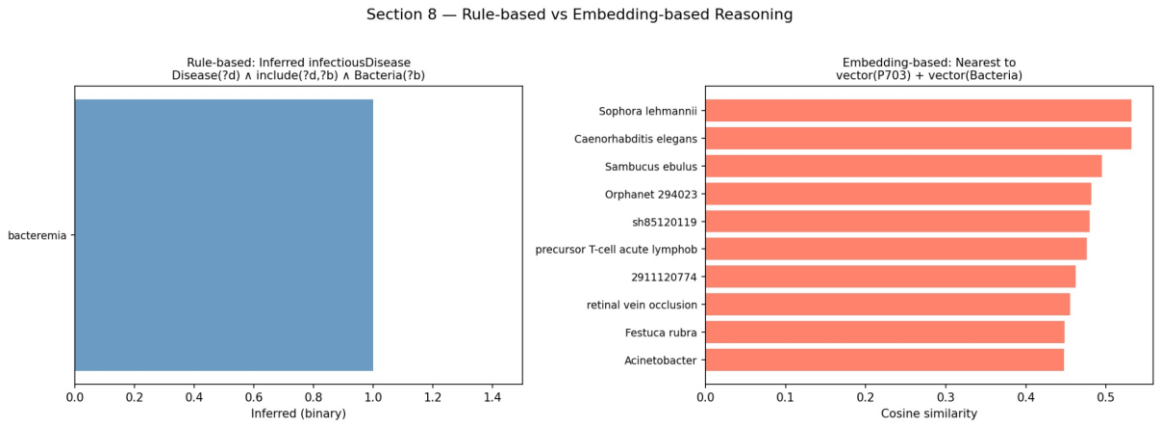

Figure 7: Section 8 comparison. The SWRL rule precisely infers bacteremia; the embedding approach (vector P703 + Bacteria) gives mixed results.

The comparison between rule-based and embedding-based reasoning highlighted fundamental complementarity between the two approaches. The SWRL rule Disease(?d) AND include(?d, ?b) AND Bacteria(?b) => infectiousDisease(?d) produced a precise, deterministic, and fully auditable result: bacteremia was correctly identified as an infectious disease. The rule is guaranteed to be correct for all entities satisfying the conditions. In contrast, the embedding-based approach attempted to replicate the same inference via vector arithmetic: vector(P703/found_in_taxon) + vector(Bacteria). While Acinetobacter appeared at rank 10 in the nearest neighbor list (a clinically relevant pathogen), the top results also included plants and genetic diseases, reflecting the noise in our KB. An important limiting factor was that our custom Sepsis-KG relations (including prop:include) were filtered out during KGE preparation due to low frequency, meaning the embedding model never learned a vector for these relations. This gap between rule coverage and embedding coverage highlights the value of a hybrid approach: SWRL rules for high-precision clinical inferences, KGE for exploratory link prediction over frequent relations.

6.6 What We Would Improve

Looking back at the project, several improvements would significantly enhance the pipeline. First, replacing spaCy's general-purpose model with scispaCy's en_ner_bc5cdr_md model, specifically trained on biomedical texts, would substantially improve NER precision for medical entity types. Second, we would adopt a URI-first modeling philosophy from the start, never storing provenance information as literals but instead as typed URI nodes, to maximize the fraction of triples usable for KGE. Third, training DistMult for 200-300 epochs (instead of 100) would likely yield further performance gains given the clear upward trend in our sensitivity analysis. Fourth, a larger LLM (7B parameters or more) would generate more reliable SPARQL queries in the RAG pipeline, reducing the need for self-repair. Finally, integrating structured biomedical databases such as DrugBank or UMLS alongside PubMed would provide higher-quality relation triples compared to unstructured text extraction.

Conclusion

This project demonstrates the feasibility of building a domain-specific medical knowledge graph from scratch using a fully automated pipeline, and connecting it to downstream reasoning and question-answering tasks. Starting from 706 documents collected from PubMed, PMC, and Wikipedia, we constructed an RDF knowledge base of 260,834 triples, achieved 100% entity alignment with Wikidata, and expanded the KB to a KGE-suitable size of 50,307 URI-URI-URI triples through targeted SPARQL queries. Our KGE experiments confirmed that KB structure directly determines which embedding model is most appropriate: DistMult's symmetric formulation achieved MRR=0.193, far outperforming TransE (MRR=0.044) on our predominantly symmetric medical ontology. The sensitivity analysis further validated the importance of KB expansion, with a +123% MRR improvement from 20k to 50k training triples. Our RAG system demonstrated the practical value of grounding LLM responses in a structured KB, improving answer accuracy from 20% (baseline) to 100% (RAG) on our evaluation set. The web chatbot interface provides an accessible demonstration of the complete pipeline, from natural language question to SPARQL-grounded answer.